

the value of $((4*5)/(2+3))$, then the equivalent s-expression in Lisp will be:

```
> (/ (* 4 5) (+ 2 3))
4
```

Evaluation

Lisp evaluates everything. It even evaluates its arguments. In Lisp, numbers evaluate to themselves—that is, any time Lisp tries to evaluate 1, the answer is always 1.

```
> 1
1
> 2
2
```

In Lisp, + is a function, and an expression like (+ 1 2) is a function call. When Lisp evaluates a function call, it does so in two steps:

1. First, the arguments are evaluated, from left to right. In the case of (+ 1 2), each argument evaluates to itself, so the values of the arguments are 1 and 2, respectively.
2. The values of the arguments are passed to the function named by the operator. In this case, it is the + function, which returns 5.

If any of the arguments are themselves function calls, they are evaluated according to the same rules. So when (/ (* 4 5) (+ 2 3)) is evaluated, this is what happens:

1. Lisp evaluates (* 4 5): 4 evaluates to 4 and 5 evaluates to 5. These values are passed to the function *, which returns 20.
2. Lisp evaluates (+ 2 3): 2 evaluates to 2 and 3 evaluates to 3. These values are passed to the function +, which returns 5.
3. The values 20 and 5 are sent to the function /, which returns 4.

Not all the operators in Common Lisp are functions, but most are. And function calls are always evaluated this way. The arguments are evaluated left-to-right, and their values are passed to the function, which returns the value of the expression as a whole. This is called the evaluation rule for Common Lisp.

Lisp is somewhat unusual in that virtually everything gets done by executing a function. In fact, there is really no more to Lisp than that. All the complexity of Lisp comes from the particular functions Lisp provides, and the details of how various types of functions are treated by the interpreter.

Call 911!

If you type something that Lisp can't understand, it will display an error message and put you into a program called a debugger. The ordinary Lisp debugger provides the ability to suspend evaluation of a form. While the evaluation is suspended (a state commonly known as a break loop), you may examine the run-time stack, check

the values of local or global variables, or change those values. The exact nature of the debugger is not part of Common Lisp, per se. Rather, this is left to the creators of various Common Lisp implementations. For example, there is no Common Lisp standard specifying the nature of the prompt to be displayed after an error. Nor is there standardisation of the commands the debugger will accept. In some implementations, the user normally does not type a command at all; rather, a special key is reserved just for this purpose.

In CLISP, to get out of the break loop and get back to the *toplevel*, just type :q as shown below:

```
[1]> (/ 1 0)

*** - /: division by zero
The following restarts are available:
ABORT          :R1      Abort main loop
Break 1 [2]> :q
[3]>
```

You must determine how the equivalent of :q has been realised in your local Lisp implementation. Experiment, check the implementation manual, or consult a local expert.

What next?

All the X-Men (movie) fans out there will agree with me when I say that the latest addition to the series—X Men First Class—has been the best yet. What could be better than learning about the origins of characters and a story line you have come to love. This, in fact, seems to be a trend in Hollywood, with movies like Star Wars, the Batman series, etc.

I am going to follow that trend, and in my third article, give you a glimpse of the evolution of Lisp. The modest beginnings of the language you have now started to love (if you are still reading this, you probably have—or is it just my charming self you cannot get enough of?!). I will also pick up the pace, as we have barely scratched the surface of Lisp. After all, plans for world domination are never fun if they take forever! 

By: Vivek Shangari

The author is a hard-core hacker, if there ever was one. He always thinks programs, and holds a piece of code in his head all the time. His favorite past time is to stop random strangers on the street and start talking about the benefits of open source over proprietary software, till they agree to switch sides, or threaten to jump off the nearest building. Every now and then, he manages to talk a bunch of unsuspecting potential programmers into letting him train them in Lisp. It's not rare to find people stumble out after the training session 'zombified', or with tears running down their faces. "Those are tears of joy!" he will patiently explain. For those of you out there who cannot be deterred, you can reach him at vivek@codeCraft.in.